

Dragon Hash Family

Implementation Self-Assessment Report

ARM Architecture — Resource Optimization Version

Meiqin Wang^{1,3,5} Kai Hu^{1,3,4} Zhuolong Zhang^{1,3} Yong Fu^{1,3,5}
Yanhong Fan^{1,3} Lixuan Wu^{1,3} Shiyao Chen^{1,3} Jiamin Cui^{1,3}
Zhongfeng Niu² Qun Liu^{1,3} Chenhao Jia^{1,3} Longzheng Cui^{1,3}

¹School of Cyber Science and Technology, Shandong University, Qingdao, China

²School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore

³State Key Laboratory of Cryptography and Digital Economy Security,
Shandong University, Qingdao, China

⁴Suzhou Research Institute, Shandong University, Suzhou, 215123, China

⁵Quan Cheng Shandong Laboratory, Jinan, China

June 30, 2026

Contents

1	Assessment Scope and Material	2
2	Algorithm Parameters	2
3	Assessment Environment and Build	2
4	Test Method and Input Data	3
5	Assessment Results	3
5.1	Functional Testing	3
5.2	Performance Testing	4
5.3	Resource Consumption and Stability	4
5.4	Transmission and Storage Overhead	4
6	Raw Evidence and Submission Checklist	5
7	Final Confirmation	5

1 Assessment Scope and Material

This report records the implementation self-assessment of the **Dragon** cryptographic hash family for the Resource Optimization Version on the ARM Architecture. It follows the *Guidelines for Self-Assessment of Cryptographic Algorithms Performance on ARM Architecture*. The assessment covers fixed-output hash computation.

Item	Description
Algorithm category	Cryptographic hash algorithm
Algorithm name	Dragon Hash Family
Sub-function	Fixed-output hash computation
Architecture	ARM
Implementation version	Resource Optimization Version
Material package	Cryptographic Hash-Dragon-ARM-Resource Optimization Version
Instances covered	Dragon-512, Dragon-768, and Dragon-1024

2 Algorithm Parameters

Instance	State (bits)	Digest (bits)	Preimage	Collision	Data limit
Dragon-512	1600	512	512	256	2^{64} bits
Dragon-768	1600	768	768	384	2^{64} bits
Dragon-1024	1600	1024	1024	512	2^{64} bits

3 Assessment Environment and Build

The operating-system, kernel, machine, hostname, and timestamp fields are taken from the ngcc JSON evidence. Processor, memory, storage, compiler, and build-tool information is recorded from the machine commands used for the assessment.

Component	Configuration
Hostname	firefly
CPU model	Rockchip RK3588
Clock frequency	2.0 GHz nominal frequency
Instruction set / machine	AArch64 / ARM 64-bit execution environment; ngcc machine field aarch64
Cores used	Single core for reported measurements
Memory / storage	16 GB RAM; 128 GB storage
Operating system	Ubuntu 22.04; kernel 5.10.160
Kernel / release	5.10.160
Compiler	GCC 9.4
Build tool	CMake 3.16.3
Third-party crypto libraries	None; the Dragon implementation is self-contained C code
Run start / end time	2026-06-30T16:02:19+0800 / 2026-06-30T16:06:05+0800

Build configuration. Resource-oriented ARM implementation using size-conscious optimization settings.

```
gcc -Os -march=armv8-a -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra
```

4 Test Method and Input Data

The assessment uses `ngcc` for functional, performance, memory, and stability testing. The script `testa64.sh` is run from `ngcc_bench-main/build`; it invokes `ngcc_bench` with `-test hash -mode all`. For performance measurements, `ngcc` uses 10000 iterations per S-level input size, exceeding the guideline recommendation of no fewer than 100 measurements.

Test item	Input data and method
Functional testing	KAT files in the submitted test-vector directories for Dragon-512 , Dragon-768 , and Dragon-1024 .
Performance testing	S1–S8 fixed input lengths: 32, 128, 512, 1024, 4096, 8192, 16384, and 65536 Bytes.
Resource testing	Static memory and peak memory reported by <code>ngcc</code> in Bytes.
Randomness	Not used for this hash-function assessment. The performance inputs and KAT files are fixed and reproducible.

KAT File Inventory

The complete KAT vector files are included in the corresponding **Self-Assessment Code and Data** package. The inventory below records the file names and sizes used by `ngcc` for functional testing.

Instance	KAT file	Size (Bytes)
Dragon-512	KAT_2_12_short.txt	2804822
	KAT_2_23_valid.txt	815
	KAT_2_33_valid.txt	824
	KAT_Loop_valid.txt	2220
Dragon-768	KAT_2_12_short.txt	3067030
	KAT_2_23_valid.txt	1007
	KAT_2_33_valid.txt	1016
	KAT_Loop_valid.txt	2284
Dragon-1024	KAT_2_12_short.txt	3333335
	KAT_2_23_valid.txt	1202
	KAT_2_33_valid.txt	1211
	KAT_Loop_valid.txt	2349

5 Assessment Results

All results below are populated from the `ngcc` JSON files listed in Section 6.

5.1 Functional Testing

Instance	Test set	Result
Dragon-512	All KAT files listed in the inventory	PASS (4104/4104, failed 0)
Dragon-768	All KAT files listed in the inventory	PASS (4104/4104, failed 0)
Dragon-1024	All KAT files listed in the inventory	PASS (4104/4104, failed 0)

5.2 Performance Testing

Instance	Size	Avg. time (ns)	Avg. cycles	Cycles/Byte	Throughput (MB/s)
Dragon-512	S1 (32 B)	2497.00	1702.11	53.19	12.29
	S2 (128 B)	2813.00	2373.15	18.54	43.73
	S3 (512 B)	3812.00	4368.94	8.53	130.54
	S4 (1024 B)	5143.00	7056.79	6.89	194.76
	S5 (4096 B)	13195.00	23209.34	5.67	307.94
	S6 (8192 B)	23848.00	44598.50	5.44	341.99
	S7 (16384 B)	45256.00	87540.31	5.34	361.20
	S8 (65536 B)	174255.00	346462.90	5.29	375.85
Dragon-768	S1 (32 B)	2497.00	1681.96	52.56	12.29
	S2 (128 B)	2764.00	2251.29	17.59	44.57
	S3 (512 B)	3908.00	4567.94	8.92	127.53
	S4 (1024 B)	5347.00	7445.07	7.27	187.74
	S5 (4096 B)	14521.00	25855.30	6.31	279.97
	S6 (8192 B)	26814.00	50533.14	6.17	304.32
	S7 (16384 B)	51161.00	99394.17	6.07	319.56
	S8 (65536 B)	198247.00	394625.62	6.02	330.40
Dragon-1024	S1 (32 B)	2490.00	1707.19	53.35	12.30
	S2 (128 B)	3091.00	2929.48	22.89	40.00
	S3 (512 B)	4928.00	6624.69	12.94	101.66
	S4 (1024 B)	7388.00	11557.86	11.29	136.62
	S5 (4096 B)	22113.00	41102.94	10.03	184.33
	S6 (8192 B)	41749.00	80549.13	9.83	195.71
	S7 (16384 B)	80953.00	159253.68	9.72	202.12
	S8 (65536 B)	317249.00	633652.95	9.67	206.51

5.3 Resource Consumption and Stability

Instance	Static memory (Bytes)	Peak memory (Bytes)	Stability
Dragon-512	77824	77824	STABLE
Dragon-768	77824	77824	STABLE
Dragon-1024	77824	77824	STABLE

5.4 Transmission and Storage Overhead

Transmission and storage-overhead metrics for public keys, private keys, signatures, ciphertexts, key-exchange messages, and key-exchange rounds are not applicable to **Dragon**, because this submission is a cryptographic hash function and does not implement KEM, signature, or key-exchange functionality.

6 Raw Evidence and Submission Checklist

Item	Evidence
KAT input directories	Dragon_ARM64/API_CryptHash/Test_Vector/Dragon512, Dragon_ARM64/API_CryptHash/Test_Vector/Dragon768, Dragon_ARM64/API_CryptHash/Test_Vector/Dragon1024
ngcc JSON result files	Dragon512s_A64_report.json.en, Dragon768s_A64_report.json.en, Dragon1024s_A64_report.json.en in Cryptographic Hash-Dragon-ARM-Resource Optimization Version/Self-Assessment Code and Data/Result
Commands and scripts	testa64.sh; manual command form documented in Self-Assessment Code and Data/README.md
Dependency declaration	Cryptographic Hash-Dragon-ARM-Resource Optimization Version/Dependency Information/DEPENDENCIES.md; no third-party cryptographic library is used
Material package contents	Algorithm Source Code, Self-Assessment Code and Data, and Dependency Information. This report is the self-assessment report for the same package.

7 Final Confirmation

This report covers Dragon-512, Dragon-768, and Dragon-1024 for the Resource Optimization Version on the ARM Architecture. The reported functional, performance, memory, and stability values are derived from the raw ngcc evidence indexed above.